

JOB PORTAL SYSTEM ARCHITECTURE (SQLITE FIRST → POSTGRESQL LATER)

1. PROJECT OVERVIEW

The Job Portal module allows:

- Job Seekers → search jobs, apply, upload resume
- Employers → post jobs, manage listings
- Authentication → JWT-based

INITIAL DATABASE: SQLite (development phase)

UPGRADE PLAN: PostgreSQL (production)

2. WHY SQLITE FIRST?

SQLite Advantages:

- Comes built-in with Django, no setup required
- Fast local development
- Easier migrations testing
- Suitable for lightweight testing phase

Use case:

→ Local dev & college project demo

3. DATABASE MIGRATION FLOW

Initial Phase:

Django Models → makemigrations → migrate → SQLite (db.sqlite3)

Later Phase (Production Upgrade):

Change DB engine → migrate existing schema → switch to PostgreSQL → upload files to cloud storage

4. DJANGO SETTINGS (SQLITE DEFAULT)

In settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

5. MODELS WITH LOCAL FILE STORAGE (MIGRATION READY)

Files stored locally & path stored in DB:

```
class Job(models.Model):
    title = models.CharField(max_length=255)
    company_name = models.CharField(max_length=255)
    city = models.CharField(max_length=50)

class JobApplication(models.Model):
    job = models.ForeignKey(Job, on_delete=models.CASCADE)
    applicant = models.ForeignKey(User, on_delete=models.CASCADE)
    resume = models.FileField(upload_to="resumes/")
    status = models.CharField(max_length=20, default="APPLIED")
    applied_at = models.DateTimeField(auto_now_add=True)
```

Run:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

This creates columns/tables in SQLite.

6. HOW FILES ARE STORED (CURRENT STAGE)

Physical location:

```
/project-root/media/resumes/filename.pdf
```

DB entry in SQLite:

```
resumes/filename.pdf
```

Files NOT stored as binary in database, only path.

7. WHEN UPGRADING TO POSTGRESQL

Modify settings.py:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'jobportal',
        'USER': 'postgres',
        'PASSWORD': 'yourpassword',
        'HOST': 'localhost',
```

```
'PORT': '5432',  
}  
}
```

Then run:

```
pip install psycopg2-binary  
python manage.py migrate
```

8. DATA MIGRATION OPTIONS

To move data from SQLite → PostgreSQL:

Option A (Simple):

- Start fresh database, run migrate only

Option B (Keep existing data):

- Use django dumpdata / loaddata

Example:

```
python manage.py dumpdata > data.json  
python manage.py loaddata data.json
```

Option C (Production Level):

- Use pgloader for full DB migration

9. FUTURE STORAGE CHANGE (FILE STORAGE)

Current:

- MEDIA folder (local disk)

Future (Recommended):

- AWS S3 using django-storages
- No database schema changes required
- FileField remains same

10. SUMMARY

DEVELOPMENT PHASE:

- SQLite + MEDIA folder
- Faster, easier setup
- DB stores file paths

UPGRADE PHASE:

- PostgreSQL + S3 Storage

- Scalable, production ready
- Same models & migrations reused

END OF DOCUMENT